

DATA SCIENCE PIPELINE

A Comprehensive Cheat Sheet based on Project Manipulations

1. Data Preprocessing & Encoding

Machine learning models require strictly numeric data. Categorical (text) data must be encoded properly without creating feature traps.

Categorical Separation & Hybrid Encoding

```
# 1. Identify text columns
text_columns = [col for col in df.columns if df[col].dtype == 'object']

# 2. Modern One-Hot Encoder setup (keeps Pandas DataFrame output)
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(sparse_output=False,
                    handle_unknown='ignore').set_output(transform='pandas')

one_hot_cols = []
for col in text_columns:
    unique_count = df[col].nunique() # MUST check singular 'col', not a list!

    if unique_count < 10:
        # Label Encoding for low-cardinality targets
        df[col] = df[col].astype('category').cat.codes
    else:
        # Queue for One-Hot Encoding
        one_hot_cols.append(col)

# 3. Apply One-Hot and Merge Safely
if len(one_hot_cols) > 0:
    encoded_df = ohe.fit_transform(df[one_hot_cols])
    df = df.drop(columns=one_hot_cols) # Drop old text columns
    df = pd.concat([df, encoded_df], axis=1) # Attach new binary columns
```

The Ambiguous Truth Value Error: If you see `ValueError: The truth value of a Series is ambiguous`, it means you passed a list of columns to an `if` statement instead of a single column (e.g., `df[['Col_A', 'Col_B']].nunique() < 10`). Pandas doesn't know how to evaluate a list of True/False conditions.

2. Feature Selection: Correlation Vector

Instead of a massive, unreadable matrix, isolate correlations strictly against your Target variable to identify the strongest predictors.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Isolate numeric columns
numeric_df = df.select_dtypes(include=['number'])

# Calculate full matrix, slice Target vector, and sort
corr_vector = numeric_df.corr()[['Target']].sort_values(by='Target', ascending=False)

# Drop self-correlation (Target vs Target = 1.0)
corr_vector = corr_vector.drop('Target', errors='ignore')

# Plot elegant 1D Heatmap
plt.figure(figsize=(5, 8))
sns.heatmap(corr_vector, annot=True, cmap='coolwarm', vmin=-1, vmax=1, fmt=".3f")
plt.title('Feature Correlation with Target')
plt.show()
```

Interpretation: Positive (> 0.4) means target increases as feature increases. Negative (< -0.4) means target decreases as feature increases. Anything near 0.0 is statistical noise and a candidate for dropping.

3. Train / Test Split

Splitting data to evaluate how the model handles completely unseen scenarios.

```
from sklearn.model_selection import train_test_split

# Drop Target using columns= parameter (vital to avoid index dropping errors)
X = df.drop(columns=['Target'])
y = df['Target']

# Strict Output Order: Train(X), Test(X), Train(y), Test(y)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Verify Shapes (Rows must match horizontally, Columns vertically)
print(X_train.shape, y_train.shape)
print(X_test.shape, y_test.shape)
```

Variable Order Bug: Swapping the output variables (e.g., `X_train, X_test, y_test, y_train`) will result in a fatal dimension mismatch error: `inconsistent numbers of samples: [102, 26]`. Python assigns variables strictly by position.

4. Cross-Validation & Model Evaluation

Evaluating multiple models to avoid overfitting, using K-Fold validation.

```

from sklearn.model_selection import cross_validate

models = {
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Logistic Regression": LogisticRegression(max_iter=1000)
}

for name, model in models.items():
    # cv=10 performs 10-Fold Cross Validation
    scores = cross_validate(model, X_train, y_train, cv=10, scoring=['accuracy', 'f1'])
    print(f"{name} Mean F1: {scores['test_f1'].mean():.4f}")

```

Testing & Classification Score Matrix

```

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

best_model.fit(X_train, y_train)
y_pred = best_model.predict(X_test)

print(classification_report(y_test, y_pred))

# Plot Confusion Matrix
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted Label')
plt.ylabel('True/Actual Label')

```

5. Core Theory & Definitions

Metric / Term	Definition & Formula
Accuracy	$(TP + TN) / \text{Total}$. The total percentage of correct predictions. Dangerous to rely on if the dataset is imbalanced.
Precision	$TP / (TP + FP)$. Out of all the "Yes" predictions the model made, how many were actually correct? (Measures False Positives).
Recall	$TP / (TP + FN)$. Out of all the real "Yes" cases in the real world, how many did the model successfully find? (Measures False Negatives).
F1-Score	$2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$. The harmonic mean. Proves the model isn't just blindly guessing the majority class.
Balanced Data	Target categories have roughly equal representation (e.g., 50 Yes / 50 No). If imbalanced, models become biased toward the majority.
Normally Distributed	A continuous numerical feature that forms a bell-curve shape. (Not to be confused with Balanced Data, which is about category counts).

6. The Dummy Classifier Baseline

A "sanity check" model that proves your algorithm actually learned something by comparing it to a strategy that just guesses the most frequent class.

```
from sklearn.dummy import DummyClassifier

dummy = DummyClassifier(strategy='most_frequent')
dummy.fit(X_train, y_train)
dummy_pred = dummy.predict(X_test)

# Compare this to your real model. A dummy on imbalanced data will
# have an F1 score of 0.00 for the minority class!
print(classification_report(y_test, dummy_pred, zero_division=0))
```